

Michael — the printer slip, and what the scanner needs

Round 29. Written after you confirmed the register now completes a sale, prints a receipt, and pops the cash drawer.

Two things in this document:

1. The answer to your question about that first receipt we printed — the one with the "dev name and address." You asked for it in this report, so it is here, first, before anything else.
2. What the Socket scanner actually needs. This is research only. **I have not written a line of scanner code yet, on purpose.** You asked me to research first so this one has fewer hiccups than the printer did, and the research turned up two things that are purchases and settings only you can do. It would be backwards to write the code and then discover them.

Part 1 — That slip with the "dev name and address"

What it is

That is the printer's **hardware self-test**. You made it by powering the printer off, holding the FEED button, and powering it back on.

I confirmed the procedure and the contents on Star's own support site, on their page "How to Print a Self-Test on Star Printers." For the TSP100III family — which is your TSP143IIIBi — the steps they publish are exactly what I had you do: power off, hold FEED, power on, let go when it starts printing.

Star describes it in one line: *"The self-test prints the printer's configuration settings."*

What is on it, per their list:

- Firmware version (printer)
- Firmware version (network card)
- Interface information
- **MAC address**
- IP address
- **Bluetooth Name**
- Communication mode
- Print settings
- Memory switch details

What the "dev name and address" actually was

That was the **Bluetooth Name** and the **MAC address**. They are the printer's own hardware identity — the name it announces itself by, and its permanent network address. It is the printer describing itself.

It is **not** a licence. It is **not** a registration. It is **not** an activation code. Nothing was used up by printing it, and nothing on it expires.

At the time, I had you print it because we were trying to establish that the printer was alive and to read its identity off it. It did that job.

Do you need to keep the paper?

No. Recycle it.

Two independent reasons, either one sufficient:

First, nothing in the register reads anything off that slip. The system identifies your counter printer by asset tag, model, and serial number — PRN-COUNTER-01 , TSP143IIIBi, serial 2550923021300119 . Those came off the sticker on the device itself, not off the self-test. The self-test is not plugged into anything we built.

Second, it is reproducible in about ten seconds. Off, hold FEED, on. That makes it a *report*, not a *record*. Records you keep. Reports you re-run when you need them.

The one line on it with any lasting use is the **Bluetooth Name**, because that is how the printer shows up in the iPad's Bluetooth list. And you can already see that on the iPad any time you want.

So: no filing cabinet needed. If you ever want it again, it is three seconds of button-holding away.

Part 2 — The scanner

First, the honest answer about the 10 seconds

You told me the scan takes about ten seconds. I want to correct something before we go further, because the obvious suspect turned out to be innocent, and if I let you keep believing it we would "fix" the wrong thing.

It is not our software waiting. I went and read the code rather than trusting my memory of it. The register finalizes the scan **the instant** the licence data is provably complete — it does not sit on a timer. There is a 1.2-second safety timer, but it only exists for the case where a scan starts and then dies halfway; on a good scan it never fires at all.

So where do the ten seconds go?

The scanner is typing. Right now your D760 is in what Socket calls Basic mode, also known as keyboard mode. It pretends to be a Bluetooth keyboard and "types" the licence into the app one character at a time. The barcode on the back of a driver's licence holds somewhere between 300 and 1,100 characters. It is sending them like a person typing, over Bluetooth.

That is the ten seconds. It is transmission, not processing.

Socket says this themselves in their documentation — that keyboard mode is "*much slower ... for barcode symbologies encoding a lot of data, such as many 2D barcodes.*" A driver's licence barcode is exactly that.

The fix is to stop it typing. In Application mode, with their real SDK, the scanner hands the whole licence over in one piece, in one go. Not 800 keystrokes — one delivery. That is the entire point of this slice, and it is why speeding up our own code would have accomplished nothing.

The good news, up front

Some things I was braced to find went the right way:

- **Your iPad's iOS version is already fine.** Socket's SDK requires iOS 15 or newer. Our app is already built for iOS 15. Exact match. No forced upgrade of anything, and no risk to the printer work we just finished.
- **The scanner itself is the right one.** The D760 is MFi certified — Apple's official accessory certification — and it lists "iOS Application Mode" as its *default* Bluetooth profile. It reads the PDF417 barcode on a driver's licence out of the box. We do not need different hardware.
- **The SDK is downloadable without any special access.** Socket's own documentation calls their package "private" and points at a portal. I checked the actual repository instead of taking their word for it, and it is public. One less gate.
- **Apple does not need to approve anything for you to use this.** Same answer as the printer. Socket requires an "MFi allow-list" registration — but only before submitting an app to the App Store. You install straight from Xcode onto your own iPad, which is not an App Store submission. It does not block you. Socket also handles that paperwork for free if the day ever comes.

The part that needs you, and needs you first

Here is the thing I want to be very direct about, because it is the difference between a smooth week and a repeat of the printer saga.

Socket's SDK will not run without an "AppKey." There is no trial mode, no degraded mode, no "works but slower." Without a key, the scanner code simply cannot start. And you cannot get an AppKey without:

1. **Buying a Socket Developer Community Membership.** It is a one-time fee, and activation is per company and for life — so this is a once-ever purchase, not a subscription. The product code is `SW1250-1463`.

I could not find the price, and I am not going to guess at it. Their product page only shows the price to a logged-in store account; the page I could read showed the product code and an "Add To Cart" button and no number. You will see the actual price at checkout. I would rather tell you "I don't know" than hand you a number I made up.

One thing worth knowing from their own page: "*After purchasing, your membership may not become active for up to one business day.*" So it is worth starting sooner rather than the morning we want to build.

2. **Generating the AppKey** once you have a Developer ID. It gets tied to our app's exact identifier, which is `com.greenwaymarijuana.register` — and it is case-sensitive, so it has to be typed exactly that way.

Two genuinely reassuring details about the key:

- **It never expires.** Generated once, good forever.
- **It works with no internet.** Socket states plainly that internet connectivity is not required to use the SDK — the key is checked on the iPad itself. If the shop's connection drops, your scanner keeps working.

The one thing to do LAST, not first

The scanner has to be switched out of keyboard mode into Application mode. You do that by scanning **one configuration barcode** — Socket publishes it in their documentation. It sticks permanently; you scan it once and it survives being powered off.

But do not do this yet.

The moment the scanner switches to Application mode, it **stops acting as a keyboard**. That is the whole point — but it also means the way the register reads it today stops working. If you flip it before the new code is on the iPad, you will have a scanner that talks to nothing, and no way to scan a licence at the counter.

So the order matters: buy and register first, I build second, and we flip the scanner over last, together with the new build. I will tell you exactly when.

There is also a small ritual for clearing the old Bluetooth pairing, which we will almost certainly need since it is currently paired as a keyboard: turn the scanner on, press the power button while holding the scan button, wait for it to switch off, and listen for **three beeps**. Three beeps means the old pairing is gone. If you do not hear three beeps, do it again — Socket's own instructions say to repeat until you hear them.

Two traps I found by reading, that would have bitten us

This is the part I want to point at, because it is the direct lesson from the printer.

In Round 26 I wrote a confident comment about how Capacitor registers plugins. I had not verified it. It was wrong, and it cost us Rounds 26, 27, and 28. So this time I went and read the actual shipped code rather than trusting what I thought I knew about this SDK. Two things fell out:

Trap one: the obvious way to check for success is wrong.

The natural way to write it — and the way **Socket's own published example code writes it** — is to check whether the result equals "no error." But their error header file says the opposite in plain text: positive numbers also mean success, just success with a warning attached. There are six such codes.

So the copy-paste-from-the-docs version would have thrown away perfectly good scans whenever one arrived with a harmless warning. Intermittently. At the counter. With a customer waiting. That is the worst kind of bug — the kind that works every time you test it.

Trap two: the convenient way to read the licence can silently return nothing.

The SDK offers a tidy "give me the barcode as text" call. Their own documentation warns, in their words, that there is "*no guaranty*" it works if the barcode is not plain text.

A driver's licence barcode is **not** plain text. It is a byte stream with invisible control characters holding the fields apart. If any part of it is not valid text, that convenient call returns **nothing at all** — not an error, just empty. The licence would vanish with no explanation.

The safe route is to take the raw bytes and convert them with a method that cannot fail. I know that now, before writing the code, instead of after three rounds of you photographing error screens.

I want to be clear that I did not know either of these from memory. I found them by opening the SDK's own files and reading them. That is the difference the standing rule makes.

One more risk I want on the record

Adding the scanner means adding entries to the app's settings file — the same file that holds the line that lets the printer talk to the iPad.

There is a list in there of accessories the app is allowed to connect to. The printer's entry is in that list right now. **If the scanner's entry replaces that list instead of being added to it, the printer stops working** — silently, with no error message, exactly the way it behaved for the three rounds we just climbed out of.

I have flagged it as the single highest-risk edit in this slice, and it is getting its own automatic check that runs on every build. I am not going to undo your printer to add your scanner.

And the promise about the fallback

If the AppKey is missing, or the SDK will not start, or anything else goes sideways — **the register must still be able to sell**. The existing keyboard- mode path stays in the code as a real, tested fallback, not as a comment claiming there is one.

A scanner problem must never be the reason a customer cannot check out. Same rule we used for the printer.

What happens next, in order

You:

1. Buy the Socket Developer Community Membership (product code `SW1250-1463`) and register. Expect up to one business day for it to activate. Tell me the price when you see it — I would like it in the record, since I could not find it.
2. Generate an AppKey for `ios:com.greenwaymarijuana.register` — exactly that, capitals and all. Send it to me along with your Developer ID.

Me, once I have those:

3. Build the scanner plugin, with both traps handled, the printer's settings protected, and the keyboard fallback kept alive.
4. Add a build check that refuses to build if the plugin is compiled but not properly connected — the exact failure that cost us Rounds 26 through 28.

Us, together, last:

5. You install the new build, then scan the Application-mode barcode and clear the old pairing. In that order.

And the standing request still stands, because it is what made Round 28 solvable: **if you get a red error naming a `.swift` file and a line number, screenshot it and send it — don't try to fix it yourself**. That is my code meeting the real world for the first time, and the full untrimmed text of it is what lets me find the cause instead of guessing at it.

Where the details live

Every technical claim in this document, with its source, is written up in `docs/slice-12-socket-capturesdk-research.md` . Each fact there cites either a file and line number in our own code, or a vendor page I actually fetched while researching. Nothing in it is from memory.